

NLP Final Project

Datasets

YelpCHI Fake Review Dataset:

<https://www.kaggle.com/datasets/virenraina/yelp-fake-review-dataset>

Ott Deceptive Opinion Spam Dataset:

<https://www.kaggle.com/datasets/rtatman/deceptive-opinion-spam-corpus>

Amazon Fake Reviews:

<https://www.kaggle.com/datasets/alexanderstephens440/fake-reviews-amazon>

Research Questions

The first research question this project hopes to answer is “Do machine learning models trained to detect deceptive opinion spam in one domain maintain their predictive power when applied to unseen domains?”

Building off of this, the secondary question hoped to be answered through this project is “Does increasing model architectural complexity improve a model's ability to generalize across domains?”

This project would help determine if there exist any consistent linguistic patterns among deceptive opinion spam, regardless of context. Furthermore, the modern technical landscape tends to favor larger, more complex models when attempting to solve generalization problems, so this project tests if architectural complexity can brute-force through domain shifts.

If this project demonstrates that greater model complexity leads to greater generalizability, it could serve as justification for companies to invest more resources into developing more complex spam detection architectures. However, if increased complexity does not improve generalization, then this would suggest that companies should instead invest resources into gathering smarter, domain-specific training data.

Methodology

This project will test three models of varying architectural complexity and see how well they complete a binary review spam classification task. The three models from lowest to highest complexity are: Logistic Regression, BiLSTM, and BERT.

Logistic Regression

The Logistic Regression model uses a Bag-of-Words approach. N-gram range is set to (1, 2), so when vectorizing the text inputs, the vectorizer is looking at unigrams and bigrams. It calculates the TF-IDF score for each token. `max_features` is set to 10,000, so the vectorizer limits the feature space to the top 10,000 tokens based on term frequency across the training dataset. Each review is transformed into an array of 10,000 TF-IDF scores.

The model then takes these TF-IDF scores and assigns a distinct weight to each score. During training, different words will get positive or negative weights depending on how frequently they appear among fake or real reviews.

After this, the model multiplies the TF-IDF score of each token by its learned weight and adds them all together along with a bias term. This numeric score is then passed through a sigmoid activation curve to compress it into a value between 0.0 and 1.0. If this score is greater than or equal to 0.5, the model classifies the review as fake, otherwise, it is classified as real.

BiLSTM

First, the `nn.Embedding` layer maps each word to a 128-dimensional vector and learns the semantic relationships between the words. This allows the model to understand meanings instead of treating each word as completely isolated.

The LSTM has three gates to ensure that relevant information is retained while processing reviews.

Because this is a bidirectional LSTM, there is an LSTM processing each review from left to right, while a second LSTM simultaneously processes each review from right to left. The model then combines the memory from both directions to ensure that the model's understanding of each word is informed by the complete context of its surroundings. The final hidden states from both the forward and backward passes are then concatenated into a single representation of the review's overall meaning.

Dropout is used to randomly zero out 30% of the connections during training in order to prevent the model from relying too heavily on any one neuron. This forces the model to learn more robust, generalized patterns of deception.

Finally, this representation of the review is passed into the `nn.Linear` layer, which assigns weights to the high-level semantic concepts extracted by the model. The memory vector is then reduced to the probability that the review is fake and the probability that the review is real.

BERT

The BERT model uses a WordPiece tokenizer to break text down into sub-words rather than whole words, allowing it to understand the roots and prefixes of words it has never seen before. It also adds a special [CLS] classification token to the very beginning of every review and a [SEP] separator token to the end.

These tokenized pieces are then passed through three simultaneous embedding layers: token embeddings, position embeddings, and segment embeddings. These vectors are added together to form a foundational starting state for each sub-word.

The data is passed through twelve stacked layers of Transformer Encoders using Multi-Head Self-Attention. This makes it so every word is compared to every other word simultaneously across multiple “heads,” which allows the model to continuously rewrite the mathematical vector for each word in order to build deep, contextual understandings based on its surroundings.

As each sequence passes through the twelve layers, the [CLS] token is forced to attend to every other word in the text. By the final layer, this token’s vector has absorbed the complete, holistic meaning of the entire review to become an overall summary vector.

Finally, the model isolates this single, dense [CLS] vector and feeds it into a linear classification layer. This layer maps the semantic representation to the probability that the review is fake and the probability that the review is real.

These models will be trained on the Amazon Fake Reviews dataset and then tested on the Ott Deceptive Opinion Spam and YelpCHI datasets. This allows for the observation of shifting domains from product reviews to hotel and restaurant reviews.

Results

When testing in-domain performance, all three models were able to successfully learn the linguistic markers of deception within the Amazon dataset. Performance increased along with model complexity, with the Logistic Regression model achieving 92% accuracy, BiLSTM achieving 93%, and BERT achieving 97% accuracy.

However, when testing across domains, the performance of all three models dropped to around 50% accuracy. All three models heavily over-predicted the truthful class and failed almost entirely to recall the fake class in the new domains.

These metrics reflect the models' performance on predicting the Fake review classification.

Model	Evaluation Domain	Accuracy	Precision	Recall	F1-Score
Logistic Regression	In-Domain (Amazon)	0.92	0.92	0.92	0.92
	Cross-Domain (Ott)	0.50	0.38	0.00	0.01
	Cross-Domain (YelpCHI)	0.51	0.65	0.02	0.05
BiLSTM	In-Domain (Amazon)	0.93	0.89	0.97	0.93
	Cross-Domain (Ott)	0.50	0.67	0.02	0.03
	Cross-Domain (YelpCHI)	0.50	0.67	0.01	0.01
BERT	In-Domain (Amazon)	0.97	0.94	1.00	0.97
	Cross-Domain (Ott)	0.51	0.75	0.02	0.04
	Cross-Domain (YelpCHI)	0.50	0.53	0.00	0.01

Analysis

Upgrading to state-of-the-art transformer architecture offered zero defense against domain shift. BERT used its immense parameter count to hyper-fit to the specific semantic context of a "product lie" (e.g., fabricating details about shipping or packaging). Because the underlying intent of deception manifests differently in experiential domains (e.g., fabricating stories about hotel staff or restaurant ambiance), the complex models were completely blinded by the change in vocabulary.

The recall represents the percentage of fake reviews each model successfully found. The recall dropped from 92-100% to 0-2% when performing cross-domain tests. This, combined with the ~50% accuracy, indicates that the models defaulted to a degenerate strategy where they defaulted to predicting almost every input as real when they could not find any of the deceptive vocabulary they learned during training.

The precision represents the percentage of predicted fake reviews that were actually fake. Despite recall dropping to 0-2%, precision for the more complex models stayed around 53-75%, while the Logistic Regression model struggled more and achieved 38% precision when testing on the Ott data. This indicates that the more complex models flagged a Yelp or Ott review as “Fake” if it contained the same specific phrasing as a fake Amazon review. On the rare occasion that the models classified a review as fake, they were sometimes correct.

Future Steps

A key next step for this project is improving data quality and consistency across domains. The results suggest that model performance is heavily dependent on the dataset itself rather than just architectural complexity. A major challenge is that high-quality, domain-specific datasets for deceptive opinion spam are difficult to obtain, and the datasets used were some of the strongest available options despite their limitations. This highlights that the challenge of generalizing across domains could also be affected by the reliability of the data within each domain. Future studies should focus not only on model complexity, but also on curating cleaner datasets, ensuring labeling consistency, and reducing noise to ensure that models are learning from reliable signals.

In addition to improving data, architectural approaches such as Domain Adversarial Neural Networks offer a way to reduce domain dependence. By adding a secondary objective that predicts the input domain and penalizing the model when it succeeds, the model is forced to learn domain-invariant features. This encourages it to focus on underlying patterns of deception rather than domain-specific vocabulary, improving its ability to generalize.

Finally, another potential direction is to shift toward large language models and prompting-based approaches. Instead of fine-tuning a classifier, zero-shot or few-shot prompting can be used to have models reason about linguistic features such as tone, detail, and consistency. These models may better handle domain variation due to their broader contextual understanding, making them a promising alternative to traditional approaches.